- Any *loopback* address, such as 127.0.0.1, which always refers to the local machine. Allowing requests to these addresses might allow access to other containers running in the same pod.
- Any *link-local* IP address, which are those starting 169.254 in IPv4 or fe80 in IPv6. These addresses are reserved for communicating with hosts on the same network segment.
- *Private-use* IP address ranges, such as 10.x.x.x or 169.198.x.x in IPv4, or *site-local* IPv6 addresses (starting fec0 but now deprecated), or IPv6 *unique local addresses* (starting fd00). Nodes and pods within a Kubernetes network will normally have a private-use IPv4 address, but this can be changed.
- Addresses that are not valid for use with HTTP, such as multicast addresses or the wildcard address 0.0.0.0.

Listing 10.15 shows how to check for URLs that resolve to local or private IP addresses using Java's `java.net.InetAddress` class. This class can handle both IPv4 and IPv6 addresses and provides helper methods to check for most of the types of IP address listed above. The only check it doesn't do is for the newer unique local addresses that were a late addition to the IPv6 standards. It is easy to check for these yourself though, by checking if the address is an instance of the `Inet6Address` class and if the first two bytes of the raw address are the values `0xFD` and `0x00`. Because the hostname in a URL may resolve to more than one IP address you should check each address using `InetAddress.getAllByName()`. If any address is private use, then the code rejects the request. Open the LinkPreviewService.java file and add the two new methods from listing 10.15 to the file.